PyDart:DP-Based Partitioning and Offline Scheduling

Parth Shinde

February 20, 2025

Contents

1 Introduction & Motivation

1.1 Overview

This codebase implements a DP-based partitioning and offline scheduling framework for accelerating DNN inference. The system partitions a DNN's FX graph into stages, each of which is scheduled on an individual node (CPU or GPU). The primary objective is to minimize the maximum execution time (*makespan*) by balancing the load across heterogeneous compute nodes.

This document provides a concise yet comprehensive overview of the library's software structure, including class responsibilities, the phased system flow from initialization to runtime, a sample design diagram, pseudocode for the full flow, essential experimental results, and details about experiment runner functions.

1.2 Motivation

- **Bottleneck Minimization:** Focuses on mitigating the slowest stage, improving overall throughput.
- **Extensibility:** A modular metric interface enables various performance measures (e.g., memory, arithmetic intensity).
- **Adaptability:** Fits both resource-constrained environments and large-scale data centers.

1.3 Scope

The document covers node management, profiling, metric interfaces, tasks, stages, the scheduling process, evaluator-based testing, and experiment runner functions (e.g., `run_experiment`, `run_multiple_experiments`). A future runtime script (for streaming or real-time inputs) may reuse these components in a similar manner.

2 Architecture Overview

2.1 System Diagram

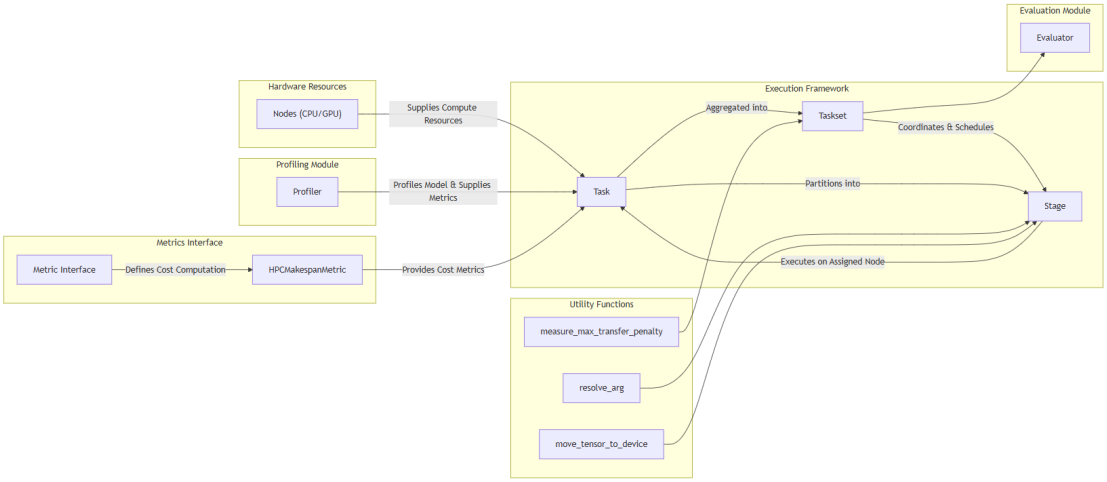

Figure 1: System Design Diagram

2.2 Component List and Responsibilities

- **Node:** Manages execution on CPU cores/GPUs, each with a worker thread and task queue.
- **Utility Functions:**
  - `resolve_arg`: Recursively resolves FX node references.
  - `move_tensor_to_device`: Moves tensors (or nested containers) to the target device.
  - `measure_max_transfer_penalty`: Measures worst-case data transfer overhead.
- **Profiler:** Uses PyTorch's profiler to gather operator-level metrics (execution time, memory usage).
- **Metric Interface and HPCMakespanMetric:** Provide a consistent way to compute cost per layer/task; default focuses on execution time.This is modifiable by the user , and he can write his own metric if required and pass it to the Taskset.
- **Task:** Encapsulates a DNN model + input data and leverages profiling data for partitioning.
- **Stage:** A contiguous block of FX nodes assigned to one node; handles data transfers and sequential node execution.
- **Taskset:** Combines tasks, coordinates stage execution across nodes, and computes global metrics.

- **Evaluator:** Compares naive (sequential) with parallel (DP-based) execution, verifying correctness and measuring speedups.

3 Detailed Component Descriptions

3.1 Node Class

Purpose & Responsibilities:

- Represents a compute unit (CPU core or GPU).
- Manages a queue of tasks (stages) with a dedicated worker thread.
- Sets/resets CPU affinity and GPU context as needed.

Key Methods:

- `assign_task`: Enqueues a function (plus args) into the node's queue.
- `stop`: Signals the worker thread to terminate.
- `discover_nodes`: Discovers available CPU/GPU nodes.

3.2 Utility Functions

resolve_arg: Recursively resolves FX node arguments using a dictionary of node outputs.

move_tensor_to_device: Moves tensors (or nested structures) to the specified device.

measure_max_transfer_penalty: Estimates the worst-case transfer time for a tensor between any two nodes.

3.3 Profiler Class

Purpose & Responsibilities:

- Instruments models using FX and torch.profiler.
- Collects per-layer execution time, memory usage, etc.
- Saves results in a CSV or DataFrame, providing data for partitioning.

Key Methods:

- `profile_model`: Profiles a model on a given node.
- `_trace_and_instrument_model`: Wraps each FX node with profiling contexts.
- `_process_profiler_data`: Aggregates raw events, updates the profiler DB.

3.4 Metric Interface and HPCMakespanMetric

MetricInterface (Abstract Base Class):

- `compute_task`: Computes overall cost for a task.
- `compute_layer`: Computes cost for a single layer on a given node.

HPCMakespanMetric:

- Implements the interface using execution time (makespan) as the principal cost driver.

3.5 Task Class

Purpose & Responsibilities:

- Represents a single DNN inference job.
- Uses profiler data and DP partitioning to split the model graph into stages.
- Manages final output and aggregated timing.

Key Methods:

- `run_offline_partition_makespan`: The DP algorithm to minimize the maximum stage cost.
- `populate_profile_records`: Gathers layer-level records from the profiler DB.
- `get_execution_order`: Topological sort of the stage DAG for sequential stage execution.

3.6 Stage Class

Purpose & Responsibilities:

- A contiguous subset of the FX graph assigned to one node.
- Handles data movement to the correct device.
- Executes each FX node in order, updating node outputs.

Key Methods:

- `run_stage`: Moves required data, runs nodes, tracks exec and transfer times.
- `add_dependency / add_dependent`: Maintains the DAG's connectivity.

3.7 Taskset Class

Purpose & Responsibilities:

- Holds multiple Task objects.
- Schedules them in parallel across available nodes.
- Computes global metrics such as makespan and utilization.

Key Methods:

- `execute_all`: Launches threads for each Task, each enqueuing stages on assigned

nodes.

- `_calculate_metrics`: Aggregates timing and load info across tasks.

3.8 Evaluator Class

Purpose & Responsibilities:

- Compares naive (sequential) vs. DP-based partitioned parallel scheduling.
- Validates correctness by comparing outputs.
- Outputs metrics like speedup, throughput, and completion times.

Key Methods:

- `run_naive_execution`: Single-device, sequential run.
- `run_parallel_execution`: Full DP-based parallel run via Taskset.
- `compare_outputs`: Ensures identical results (within tolerance).
- `analyze_speedup_throughput`: Prints overall performance gains.

4 Experiment Runner Functions

Purpose: Simplify the creation and execution of test scenarios for evaluating different hardware setups, heavy/light model mixes, and scheduling strategies.

- `run_experiment(k, heavy_light_ratio, num_tasks)`:
  - Discovers and initializes k nodes.
  - Creates `num_tasks` tasks, dividing them into heavy vs. light according to the ratio.
  - Profiles each task on each node, populates partition data, and forms a Taskset.
  - Uses an Evaluator to run naive and parallel executions, prints metrics, then cleans up nodes.
- `run_multiple_experiments(k, num_tasks)`:
  - Loops over various heavy/light ratios (e.g., 0.1 to 0.9).
  - Calls `run_experiment` for each ratio.
  - Logs speedup, throughput, and makespan, optionally plotting or saving the data.

5 Phased System Flow

5.1 Initialization Phase

1. **Node Discovery:** Call `discover_nodes` to find CPU/GPU resources; spawn `Node` objects.
2. **Profiler Setup:** Instantiate the Profiler (`mode="init"`).
3. **Task Creation:** Models + input data → `Task` objects.
4. **Profiling:** `profile_model` runs each model on each node; timing data is saved in CSV.
5. **DP Partitioning:** For each task, call `run_offline_partition_makespan` to form stages.

5.2 Evaluation Phase

1. **Experiment Setup:** Possibly use `run_experiment` to define heavy/light mixes, number of tasks, etc.
2. **Taskset Creation:** Aggregate tasks + nodes in a `Taskset`.
3. **Naive vs. Parallel:** Evaluator runs each approach, gathering performance metrics.
4. **Comparison:** Use `analyze_speedup_throughput` to measure speedup, throughput, and makespan differences.

5.3 Runtime Phase

1. **Execute All:** `taskset.execute_all()` spawns a thread per Task, each assigning its stages to the assigned Node's queue in topological order.
2. **Worker Threads:** Each Node processes queued stages in a FIFO manner, respecting data transfers and dependencies.
3. **Output Collection:** When the final stage completes, each Task's result is available.
4. **Adaptation (Optional):** If workloads or arrival times change, re-profile or repartition tasks before repeating.

6 Pseudocode for the Full Flow

```
# Initialization Phase
nodes = Node.discover_nodes(disjoint=True)
profiler = Profiler(mode="init")
tasks = []

for i in range(num_tasks):
    model = create_model(i)            # e.g., heavy or light
    input_data = create_test_data(...)
    task = Task(task_id=f"task_{i+1}",
                model=model,
                input_data=input_data,
                profiler=profiler)
    tasks.append(task)

# Profile tasks on each node
for t in tasks:
    for node in nodes:
        profiler.profile_model(t.model, t.input_data, node, t.task_id)

for t in tasks:
    t.populate_profile_records()
    t._available_nodes = nodes
    t.run_offline_partition_makespan()

# Evaluation Phase
taskset = Taskset(tasks, nodes)
evaluator = Evaluator(taskset, profiler)
evaluator.run_naive_execution()
evaluator.run_parallel_execution()
evaluator.compare_outputs()
evaluator.analyze_speedup_throughput()

# Runtime Phase
# If directly running tasks in parallel after partitioning:
taskset.execute_all()     # concurrent execution
for node in nodes:
    node.stop()           # cleanup
```

7 Experimental Results

7.1 Setup

Experiments often involve mixing heavy (e.g., `PretrainedResNet18`) and light (e.g., `SimpleCNN`) tasks, distributing them across 2–8 compute nodes. The code measures speedup, makespan reduction, and throughput gains when using DP partitioning compared to naive sequential execution.

7.2 Findings

- **Speedup:** Typically achieves around 1.8× (80% faster) over naive, reaching up to 2–3× in heavier-task scenarios.
- **Makespan Reduction:** Large improvements in total completion time, particularly when 70–90% of tasks are heavy.
- **Throughput Gain:** Effective concurrency ensures multiple tasks run simultaneously, leveraging hardware fully.

7.3 Example Usage

Running `run_multiple_experiments(k=2, num_tasks=20)` explores different heavy-to-light ratios, logs speedups, and can plot the results (makespan vs. ratio, throughput vs. ratio).

8 Future Work: Runtime Script

- The same DP-based partitioning can be extended to handle real-time or streaming arrivals.
- A runtime script could dynamically schedule newly arriving tasks by reusing the existing partitioning logic.

9 Conclusion

- **Summary:** This DP-based framework effectively splits DNN models (FX graphs) across multiple CPU/GPU nodes, reducing makespan and improving throughput.
- **Modularity:** From Node and Profiler to Taskset and Evaluator, each component is designed for extensibility (custom metrics, new scheduling policies, etc.).
- **Performance Gains:** Empirical tests show consistent 1.8–3× speedups over naive sequential runs, making it useful for HPC, batch, or potentially real-time deployments.